

Day 4 – Authentication & Permissions

Goals: Lock down the API with token auth and owner-level permissions.

Add to `INSTALLED_APPS`:

```
'rest_framework.authtoken',
```

```
python manage.py migrate
```

Update `settings.py`:

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.TokenAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticatedOrReadOnly',
    ],
}
```

`games/permissions.py` — custom permission so publishers can only edit their own games:

```
from rest_framework.permissions import BasePermission, SAFE_METHODS

class IsOwnerOrReadOnly(BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.method in SAFE_METHODS:
            return True
        return obj.publisher.user == request.user
```

Add a user registration endpoint in `games/views.py`:

```
from django.contrib.auth.models import User
from rest_framework.authtoken.models import Token
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import AllowAny
from rest_framework.response import Response
from rest_framework import status

@api_view(['POST'])
@permission_classes([AllowAny])
def register(request):
    username = request.data.get('username')
    password = request.data.get('password')
    if not username or not password:
        return Response({'error': 'Username and password required.'}, status=status.HTTP_400_BAD_REQUEST)
    user = User.objects.create_user(username=username, password=password)
    token, _ = Token.objects.get_or_create(user=user)
```

```
return Response({'token': token.key}, status=status.HTTP_201_CREATE
```

Add to `urls.py`:

```
from games.views import register
urlpatterns += [path('api/register/', register)]
```

Instructor Teaching Plan

Time Breakdown (90 min)

Segment	Duration	Activity
Auth concepts	15 min	Compare session auth (cookie-based, browser), token auth (header-based, API), and JWT (self-contained). Explain why token auth is the right choice for a stateless API.
Token auth setup	15 min	Add <code>rest_framework.authtoken</code> to <code>INSTALLED_APPS</code> , run <code>migrate</code> , update <code>REST_FRAMEWORK</code> settings. Generate a token via <code>manage.py shell</code> to verify.
Custom permission class	20 min	Write <code>IsOwnerOrReadOnly</code> . Explain <code>has_permission</code> (request-level) vs <code>has_object_permission</code> (object-level). Explain <code>SAFE_METHODS</code> . Apply to <code>GameViewSet</code> .
Registration endpoint	20 min	Write the <code>register</code> view together. Explain <code>create_user</code> (hashes password) vs <code>create</code> (stores plaintext — never do this). Show <code>Token.objects.get_or_create</code> .
End-to-end auth test	10 min	Register a user → get token → use token in <code>Authorization: Token ...</code> header → hit a protected endpoint. Try without the header → confirm 401.
Q&A + checkpoint	10 min	

Key Concepts to Introduce

- **Authentication vs Authorization** — authentication asks "who are you?", authorization asks "what are you allowed to do?". Token auth handles the first; permission classes handle the second.
- **`IsAuthenticatedOrReadOnly`** (global default) — unauthenticated users can `GET`; only authenticated users can `POST/PATCH/DELETE`. Good starting default for a public catalog.
- **`BasePermission`** — the base class for all DRF permissions. Override `has_permission` for request-level checks, `has_object_permission` for instance-level checks. Both must return `True` for access to proceed.
- **`SAFE_METHODS = ('GET', 'HEAD', 'OPTIONS')`** — read-only HTTP methods. The pattern "allow reads for all, writes only for owner" is idiomatic DRF.
- **`User.objects.create_user()`** — hashes the password via Django's password hasher. `create()` skips hashing. Always use `create_user` for real users.
- **`@permission_classes([AllowAny])`** — explicitly bypasses the global default for a specific view. Required for registration — the user doesn't have a token yet.

Common Mistakes to Address

- Using `create()` instead of `create_user()` — password stored in plaintext, authentication will always fail.
- **Missing `rest_framework.authtoken` migration** — Token model doesn't exist yet, Django raises `ProgrammingError`. Always run `migrate` after adding the app.
- **Wrong `Authorization` header format** — must be `Token <token>` with a space (not `Bearer`, not `token`). Confuses students coming from JWT backgrounds.
- **`has_object_permission` not called for list actions** — DRF only calls `has_object_permission` when the view retrieves a specific object (retrieve, update, destroy). It's never called for `list` or `create`.
- **`@permission_classes` not overriding the global default** — students add `@permission_classes([AllowAny])` but forget `@api_view(['POST'])`, so Django treats it as a regular function view and DRF never runs.

? Check for Understanding

"What's the difference between `has_permission` and `has_object_permission`? Give an example where you'd use each."

Answer: `has_permission` checks if the user has access to the endpoint in general (checked at the start of the request, e.g., "Is the user logged in?"). `has_object_permission` is only called during detail views (retrieve/update/delete) to check access to a specific database object (e.g., "Is the logged-in user the owner of this game record?").

"Why do we use `create_user()` instead of `create()` when making `User` objects?"

Answer: `create_user()` is a helper method on Django's `User` manager that handles password hashing using a secure algorithm (like PBKDF2). Using `create()` directly would write the password to the database as plaintext, exposing it to database administrators or security breaches, and preventing the user from logging in since Django's auth system expects a hashed password.

"If we didn't add `@permission_classes([AllowAny])` to the `register` view, what would happen when a new user tries to register?"

Answer: The registration request would be blocked by the global permission default (`IsAuthenticatedOrReadOnly`), returning a `401 Unauthorized` or `403 Forbidden` response. Since a new user does not have a token yet, they would be unable to register.

"What does `IsAuthenticatedOrReadOnly` allow vs deny? Is it the right default for this API?"

Answer: It allows anyone (authenticated or anonymous) to perform safe, read-only HTTP methods (`GET`, `HEAD`, `OPTIONS`), but restricts writing methods (`POST`, `PUT`, `PATCH`, `DELETE`) to authenticated users. It is an excellent default for this API because it keeps the game catalog publicly browsable while securing modifications and purchases.

✓ Day Checkpoint

`POST /api/register/` returns a token. `POST /api/games/` without the token returns

401. POST /api/games/ with Authorization: Token <token> returns 201. PATCH /api/games/1/ with a token from a *different* user returns 403.